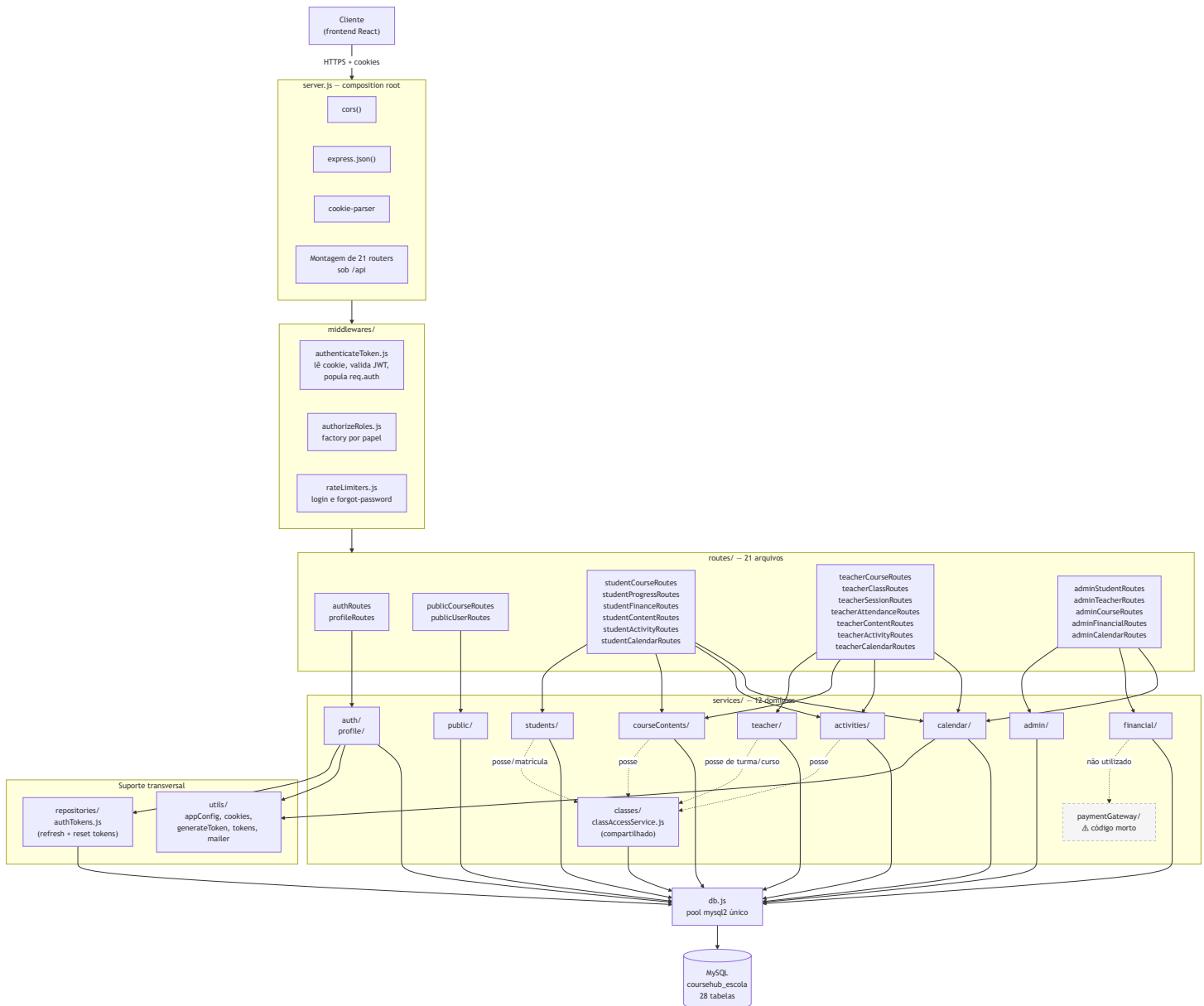


Diagrama — Arquitetura do Backend

Reflete o estado pós-padronização de 2026-08-02: `server.js` como composition root de 101 linhas, 21 routers, 12 domínios de service. Nenhum módulo abaixo é hipotético — todos existem em `backend/` na data deste documento.



Que problema este diagrama esclarece

Responde "onde eu mexo se preciso mudar X" para qualquer pessoa nova no backend. Antes da padronização de 2026-08-02, essa pergunta não tinha uma resposta consistente — dependendo do domínio, a resposta era "em `server.js` , em algum lugar entre as linhas 1.000 e 9.000" ou "no arquivo

de service certo". Hoje a resposta é sempre a mesma forma: existe um `routes/<domínio>Routes.js` fino e um `services/<domínio>/*.js` com a lógica.

Como interpretar os elementos

- **Setas sólidas** (`-->`) são dependências diretas de chamada (router chama service; service chama o pool de conexão).
- **Setas tracejadas** (`-.>`) marcam duas coisas distintas: dependência de um serviço compartilhado (`classAccessService.js` , usado por quatro domínios) e uma dependência que **deveria** existir mas não existe (`financial` $\rightarrow$ `paymentGateway` , marcado "não utilizado" porque o pagamento manual grava `gateway: "simulated"` direto, sem passar pelo módulo).
- O nó `paymentGateway/` recebe a mesma marcação visual (cinza tracejado) usada nos outros diagramas para "existe mas não é usado" — consistência proposital entre os 9 diagramas deste conjunto.
- Cada domínio de `services/` mapeia exatamente a uma subpasta real de `backend/services/` — não há agrupamento artificial para simplificar o diagrama.

Que decisões arquiteturais este diagrama evidencia

- **Um único pool de conexão MySQL** (`db.js`), compartilhado por todos os services — não há pool por domínio nem por transação. Toda transação de negócio (ex. `criar aluno = users + students` em uma única operação atômica) usa `db.promise().getConnection()` para obter uma conexão dedicada dentro do service, não um pool separado.
- `classAccessService.js` é o **único módulo verdadeiramente transversal** — a decisão consciente foi não criar uma camada de "core" ou "shared" maior; só o que é genuinamente reusado por 4+ domínios (resolver posse de turma/curso/matricula) foi extraído.
- **Repositories existem só para tokens de autenticação** — o projeto não adotou um padrão de repository genérico para todas as entidades; `authTokens.js` é uma exceção deliberada porque a lógica de hash/expiração de token é sensível o suficiente para justificar isolamento.

Trade-offs que esta arquitetura representa

- **Cada service define seu próprio `createServiceError`** em vez de importar um helper de erro compartilhado — gera pequena duplicação (a mesma função de 5 linhas em ~20 arquivos), mas evita que todo o backend dependa de um único módulo de erro central, o que facilitaria acoplamento acidental entre domínios que deveriam ser independentes.

- **Sem camada de "controller" separada do router:** a rota Express é o controller. Isso mantém o backend enxuto (menos um nível de indireção), mas significa que qualquer lógica de formatação de resposta HTTP-específica (status code, shape do JSON de erro) vive espalhada em cada arquivo de rota, não centralizada.
- **paymentGateway/simulatedGateway.js como código morto tolerado:** em vez de removê-lo nesta rodada de limpeza, foi deixado documentado como fora de escopo — decisão de manter o blast radius da limpeza de 2026-08-02 controlado (ver ADR-001), não uma recomendação de mantê-lo indefinidamente.

Como este diagrama deve evoluir

Ao adicionar um novo domínio de negócio, ele deve nascer diretamente como um par `routes/<nome>Routes.js` + `services/<nome>/`, nunca inline em `server.js` — qualquer PR que adicione lógica de negócio a `server.js` deve ser tratado como um desvio da arquitetura documentada aqui, não uma variação aceitável. Se `paymentGateway/simulatedGateway.js` for finalmente conectado ou removido, atualize a aresta tracejada correspondente e a marcação visual.